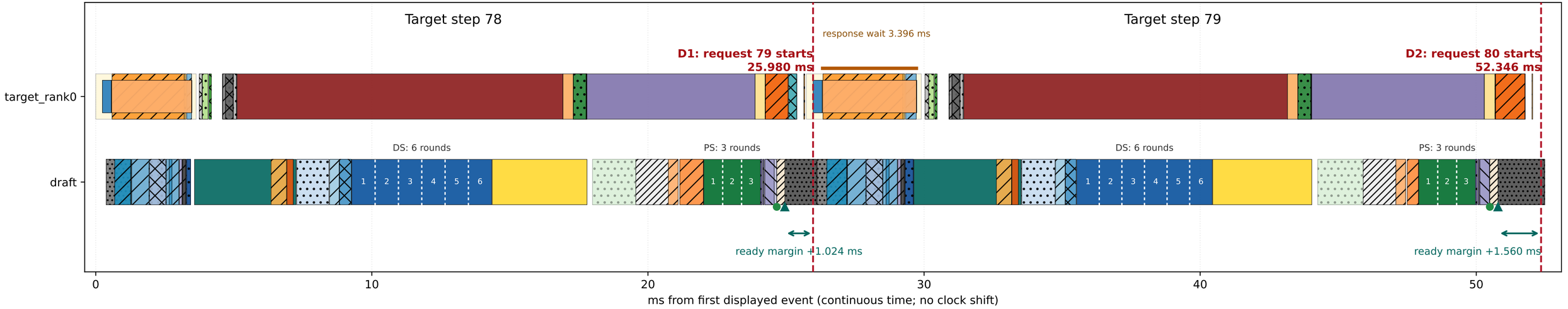
Previously discussed example | Llama2-7B + llama-68M | UID 244_t0 | steps 78-79

Historical v1.0 82d964e0 | RTX3090 | B1 BF16 T0.7 | exit20 K1/K2 6/3 M1/M2 12/8 C3 | not a new ver1.2 performance run

Red dashed line = next target request starts (observed deadline reference). Green circle = PS cache ready; triangle = draft ready to receive.

Both target steps and their DS/PS work share one time axis. Only the following request boundary is used for the second deadline.



Prepared in step	Cache ready (ms)	Receive-ready (ms)	Next send (ms)	Cache margin (ms)	Ready margin (ms)	Next send span (ms)	Next response wait (ms)
78	24.665	24.957	25.980	+1.316	+1.024	0.329	3.396
79	50.492	50.786	52.346	+1.854	+1.560	0.325	2.313

Margin = next send start minus readiness. Positive means ready before the observed request; negative means preparation ran past it.

This is not a causal stall estimate or a promise that all margin can be used for more forwards. The second next-response wait is outside the two-step view.

Command-to-request gap (unprofiled)
PS dispatch gap (unprofiled)
PS prepare A: roots / visibility
PS prepare B: graph input buffers
Post-PS gap (unprofiled)
draft_recv_cmd
draft_recv_request
draft_response_pack
draft_send_fused
draft_send_parent_q_p1
draft_send_response
exit_logits
exit_proxy_launch
exit_proxy_side
final_logits
glue
graph_post
graph_pre
hit_cache_respond_hit_k1
merge_cache
p1_graph_replay
p1_prepare
p1_root_build
p1_slot_prepare
p2_cache_merge
p2_graph_replay
p2_output_convert
phase2_build
proxy_send_enqueue
proxy_wait
target_postprocess
target_recv_fused
target_recv_parent_q_p1
target_recv_response_wait
target_response_meta_read
target_response_received
target_send_request
target_spec_wait
tree_accept_walk
tree_hit_pack_served_p1
tree_hit_parent_q_gather_p1
tree_hit_rerank_p1
tree_kv_commit
tree_kv_restore
tree_parent_q_select
tree_prep
tree_proxy_graph_replay
tree_proxy_topology_prepare
tree_replay
tree_verify_attention_buffers
tree_verify_mask_prepare
tree_verify_meta_cpu
tree_wire_parse_validate
verify_accept_prep
verify_sample_accept
verify_setup

Hatched gaps = unprofiled elapsed intervals, NOT proven GPU idle. Nested spans are not additive. White graph dividers show counts, NOT per-forward measurements.
Raw intervals retained, no 0.02 ms visibility floor. Cross-device margins inherit CUDA-to-host clock alignment uncertainty; exact raw events and source hashes accompany this figure.